Automatização de Configurações - IaC

Infraestruturas de Computação na Nuvem, Aula 07

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

Objetivos de Aprendizagem

- Justificar a automatização da configuração e do provisionamento face à gestão manual de infraestrutura.
- Distinguir abordagens declarativas e imperativas e explicar o conceito de idempotência.
- Descrever a arquitetura e os componentes do Ansible.
- Explicar o funcionamento do Terraform, em particular o state file e o ciclo plan/apply.
- Diferenciar provisionamento de gestão de configuração.
- Aplicar boas práticas de gestão de segredos e CI/CD a infraestrutura.

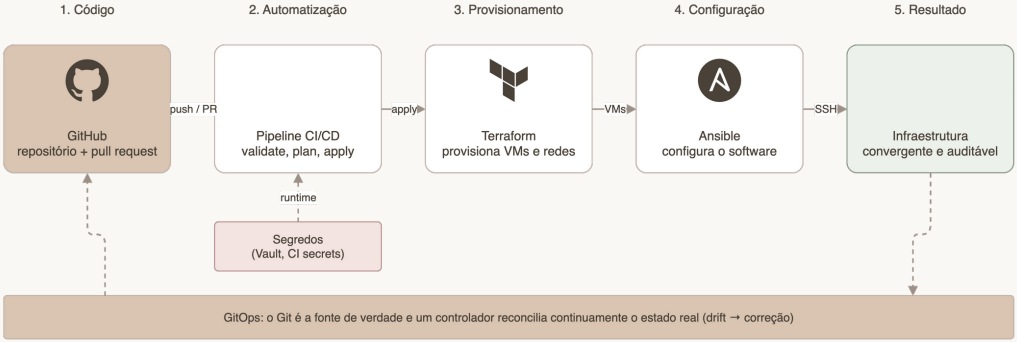
Agenda (1/2)

- DevOps e a tensão entre desenvolvimento e operações.
- Limitações da configuração manual.
- Infrastructure as Code (IaC).
- Abordagens declarativa e imperativa; idempotência.
- Gestão de configuração com Ansible.

Agenda (2/2)

- Provisionamento declarativo com Terraform e gestão do state file.
- Provisionamento versus gestão de configuração.
- Gestão de segredos.
- CI/CD aplicado a infraestrutura.

Visão Geral da Aula

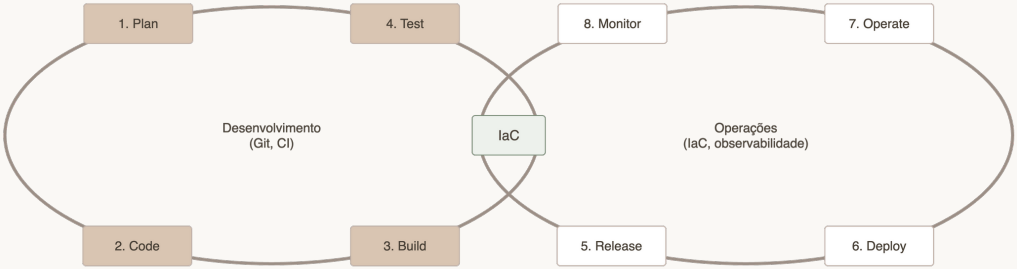


Da alteração ao código até à infraestrutura em produção, sem intervenção manual.

DevOps e a Tensão Entre Desenvolvimento e Operações

- As equipas de **Dev** (evolução do software) e de **Ops** (estabilidade da infraestrutura) têm incentivos historicamente opostos: maximizar a frequência de alterações versus minimizar o risco de falha.
- A separação organizacional gera handoffs manuais, ambientes inconsistentes e lead times elevados entre commit e produção.
- A resposta burocrática (aprovações manuais, janelas de alteração rígidas) aumenta o lead time sem melhorar a qualidade.
- **DevOps** designa a cultura e o conjunto de práticas que integram as duas funções num único ciclo, assente em versionamento, integração e entrega contínuas, e automatização.

O Ciclo DevOps



Ciclo DevOps: a infraestrutura descrita como código fecha o ciclo entre a entrega (Dev) e a operação (Ops).

Cenários de Aplicação do DevOps

- Na integração contínua, cada commit desencadeia a compilação e a execução automática de testes.
- Na entrega e implantação contínuas, as versões seguem para produção de forma frequente e reversível.
- As implantações progressivas (canary, blue-green) e as feature flags limitam o impacto de cada alteração.
- A criação de ambientes efémeros de teste por pull request exige infraestrutura provisionada a pedido.
- A observabilidade (métricas, logs, alertas) alimenta o planeamento do ciclo seguinte.
- O DevSecOps integra no pipeline a análise de dependências e a deteção de segredos expostos.

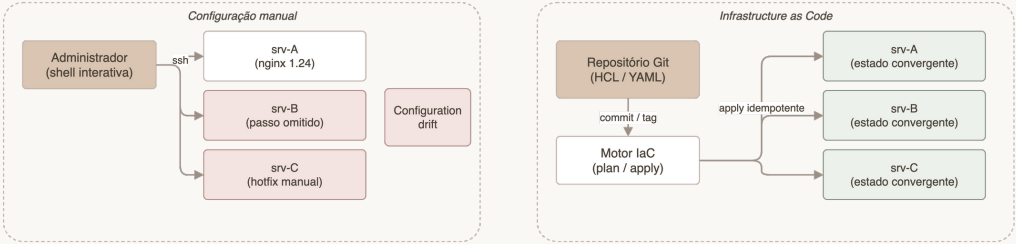
Relevância para a Unidade Curricular

O DevOps surge nesta unidade curricular porque a automatização da infraestrutura é a condição técnica que permite aos cenários anteriores serem executados de forma contínua, repetível e auditável.

Limitações da Configuração Manual

- Não escala: o esforço cresce linearmente com o número de servidores.
- O **configuration drift** resulta da evolução independente de cada servidor e é difícil de detetar sem inventário do estado real.
- O erro humano (passos omitidos, comandos mal aplicados) gera inconsistências difíceis de diagnosticar.
- Os ambientes de desenvolvimento, teste e produção divergem, pelo que os testes deixam de ser representativos.
- A recuperação de desastres é lenta, pois recriar a infraestrutura manualmente demora horas ou dias (RTO elevado).
- A auditoria e a conformidade exigem registo de quem alterou o quê e quando.

Configuração Manual e Infrastructure as Code



Mesmo objetivo, resultados diferentes: a automatização elimina a divergência entre servidores.

Infrastructure as Code (IaC)

- A infraestrutura (servidores, redes, serviços) é descrita em ficheiros de texto (YAML, HCL) e não através de passos manuais.
- O código é versionado em Git, o que permite histórico, revisão por pares (code review) e rollback para qualquer versão.
- A **idempotência** garante que aplicar a mesma definição várias vezes converge sempre para o mesmo estado.
- A **repetibilidade** permite gerar ambientes idênticos em qualquer momento e testar alterações antes de produção (lint, dry-run).
- O código constitui documentação atualizada e reduz o tempo de provisionamento de dias para minutos.

Do Princípio às Ferramentas

- O IaC é um princípio e não uma ferramenta: define que a infraestrutura deve ser descrita, versionada e aplicada como código.
- A sua concretização envolve dois problemas distintos.
- O primeiro consiste em garantir que os recursos existem (VMs, redes, storage), o que corresponde ao **provisionamento**, abordado pelo **Terraform**.
- O segundo consiste em definir o que corre dentro de cada recurso (pacotes, ficheiros, serviços), o que corresponde à **gestão de configuração**, abordada pelo **Ansible**.
- Em ambos os casos aplicam-se os mesmos fundamentos, a descrição declarativa do estado e a idempotência.

Ansible e Terraform

- O **Ansible** é uma ferramenta open source de gestão de configuração e automatização, que aplica por SSH, sem agente, o estado descrito em playbooks YAML a máquinas já existentes.

- O **Terraform** é uma ferramenta open source de provisionamento declarativo, que cria, altera e destrói recursos de infraestrutura (VMs, redes, storage) através das APIs dos providers, a partir de código HCL.

Abordagens Imperativa e Declarativa

- Na abordagem **imperativa** descreve-se a sequência de passos (por exemplo, um script shell), pelo que o resultado depende do estado inicial.
- Na abordagem **declarativa** descreve-se o estado final e a ferramenta calcula as ações necessárias, tal como no Kubernetes (Aula 05).
- Esta abordagem torna possível a **idempotência**: reaplicar a definição converge para o mesmo estado, e as ferramentas reportam o que mudou (*changed*) e o que já estava conforme (*ok*).

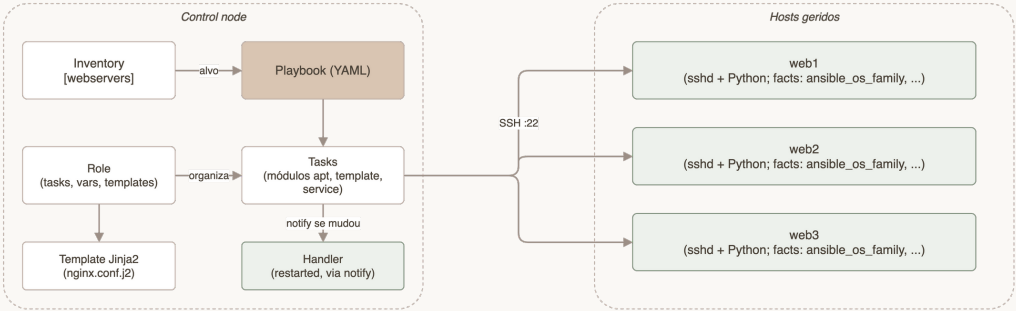
Ansible

- Comunica por SSH (porta 22) e não exige agente: cada módulo é transferido para o host, executado com Python e removido.
- Os playbooks são escritos em **YAML** e descrevem o estado desejado de forma declarativa.
- O **inventory** lista os hosts geridos, agrupados por função.
- Os módulos (`apt`, `copy`, `service`) são idempotentes.
- O Ansible é mantido pela Red Hat e distribuído como software open source.

Componentes de um Projeto Ansible

- Um **playbook** é um ficheiro YAML que associa um conjunto de tasks a um grupo de hosts.
- Uma **task** é uma ação individual, executada por um **módulo**.
- Um **handler** é uma task executada apenas quando outra notifica que algo mudou (por exemplo, reiniciar o serviço após alterar a configuração).
- Uma **role** organiza tasks, variáveis, templates e ficheiros numa unidade reutilizável.
- Um **template Jinja2** é um ficheiro de configuração parametrizado por variáveis, instanciado para cada host.
- Os **facts** são informação recolhida automaticamente sobre cada host (sistema operativo, memória, endereços IP).

Estrutura de um Projeto Ansible



ansible-playbook -i inventory site.yml: cada task é um módulo transferido e executado por SSH.

Exemplo de Playbook

```
- hosts: webservers
  become: true
  tasks:
    - name: Instalar o Nginx
      ansible.builtin.apt:
        name: nginx
        state: present
    - name: Copiar a configuração
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      notify: Reiniciar o Nginx
  handlers:
    - name: Reiniciar o Nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

Numa segunda execução sem alterações, todas as tasks reportam *ok* e o handler não é executado.

Terraform e Provisionamento Declarativo

- O Terraform é uma ferramenta declarativa para provisionar recursos de infraestrutura (VMs, redes, storage).
- Os **providers** são plugins que traduzem o código em chamadas às APIs de clouds ou plataformas específicas.
- O **state file** regista o que foi efetivamente criado, para o comparar com o estado desejado.
- O fluxo `plan` / `apply` mostra primeiro as alterações previstas e só depois as executa.
- A linguagem usada é a HCL (HashiCorp Configuration Language).

Exemplo de Configuração Terraform

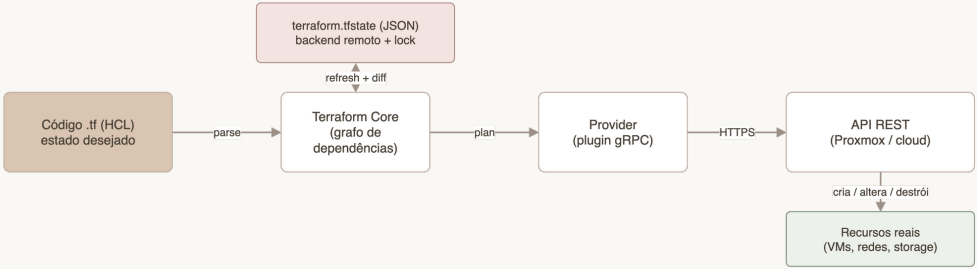
```
terraform {
  required_providers {
    proxmox = { source = "bpg/proxmox" }
  }
}

variable "vm_count" { default = 2 }

resource "proxmox_virtual_environment_vm" "web" {
  count     = var.vm_count
  name      = "web-${count.index + 1}"
  node_name = "pve1"
  cpu    { cores = 2 }
  memory { dedicated = 2048 }
}
```

Alterar `vm_count` para 3 e executar `terraform plan` indica a criação de uma única VM adicional.

Terraform, State File e Providers

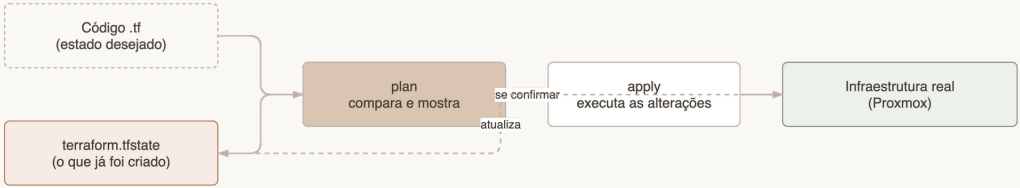


O state file liga os recursos descritos no código aos recursos reais; deve ser tratado como dado sensível.

O State File

- O **state file** regista o mapeamento entre os recursos descritos no código e os recursos reais.
- Permite ao Terraform determinar, em cada execução, o que já existe, o que falta criar e o que deve ser destruído.
- Em equipa, o estado deve residir num **backend remoto** partilhado, com **locking** para evitar execuções simultâneas.
- O ficheiro pode conter dados sensíveis em texto simples e deve ser tratado como um segredo.
- Alterações manuais fora do Terraform introduzem drift e serão revertidas na aplicação seguinte.

O Ciclo Plan / Apply



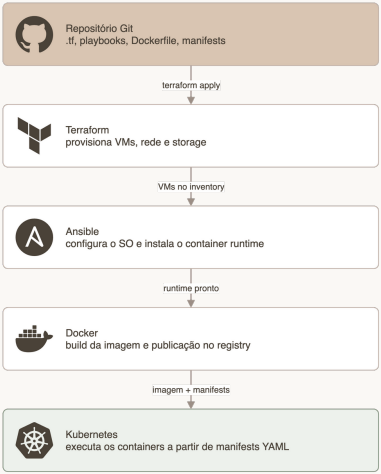
Alterações feitas à mão fora do Terraform divergem do estado e são revertidas na aplicação seguinte.

Provisionamento e Gestão de Configuração

- O **provisionamento** decide que máquinas e recursos existem (por exemplo, Terraform).
- A **gestão de configuração** decide o que corre dentro de cada máquina (por exemplo, Ansible, Puppet).
- Fluxo habitual: o Terraform cria a VM e o Ansible instala e configura o software.
- As duas camadas são complementares e não concorrentes.

Integração com Docker, Kubernetes, Terraform e Ansible

- O Terraform provisiona VMs, rede e storage, incluindo os nós do cluster.
- O Ansible configura esses nós e instala o container runtime.
- O Docker empacota a aplicação numa imagem imutável, publicada num registry pelo CI/CD.
- O Kubernetes executa a imagem de forma declarativa, a partir de manifests YAML.
- Tudo é versionado no Git e aplicado pelo mesmo pipeline.

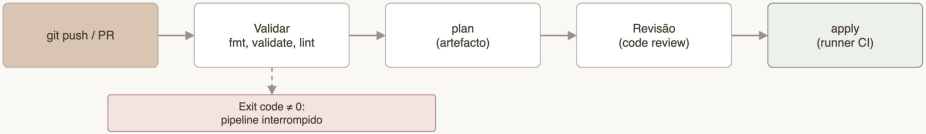


Gestão de Segredos

- O código de infraestrutura é versionado em Git, mas passwords, chaves e tokens não podem ser guardados no repositório em texto simples.
- O **Ansible Vault** cifra ficheiros de variáveis, permitindo guardá-los no repositório.
- O **HashiCorp Vault** e serviços equivalentes armazenam segredos de forma centralizada e fornecem-nos às ferramentas apenas em tempo de execução.
- As variáveis de ambiente podem ser injetadas pelo sistema de CI/CD, sem nunca serem escritas no repositório.
- Um segredo que tenha sido alguma vez submetido (commit) deve ser considerado comprometido e rodado.

CI/CD para Infraestrutura

- Aplicam-se ao código de infraestrutura os mesmos princípios de integração e entrega contínuas usados no software.
- O pipeline valida a sintaxe, gera o plano (`plan` ou dry-run), submete-o a revisão e só então executa o `apply`.
- Os testes automatizados antecedem qualquer alteração em produção, o que reduz o risco de alterações manuais não controladas.



Nenhuma alteração chega a produção sem validação, plano revisto e aprovação.

Síntese de Conceitos de Infraestrutura

Conceito	Definição técnica	Exemplo
Idempotência	$f(f(x)) = f(x)$: reaplicar a definição não altera o estado	`state: present`
Configuration drift	Divergência entre o estado real e o declarado	Edição manual de `nginx.conf`
Declarativo	Especifica-se o estado final; o motor calcula o delta	Playbook, ficheiro `.tf`
State file	Mapeamento entre recursos declarados e reais	`terraform.tfstate`
Agentless / agent-based	Push por SSH versus pull por agente residente	Ansible; Puppet, Chef
Imutabilidade	Substituição da instância em vez de alteração in-place	Imagem Packer / Docker
Reconciliação	Ciclo contínuo observar, comparar e corrigir	Argo CD, Flux

Síntese da Aula (1/2)

- O DevOps responde à tensão entre Dev e Ops através de cultura, automatização e versionamento.
- A configuração manual conduz a drift, erros e falta de reprodutibilidade.
- O IaC trata a infraestrutura como código versionado, idempotente e repetível.
- O Ansible (agentless, SSH, YAML) contrasta com Puppet e Chef (agent-based), que mantêm um agente em cada host.

Síntese da Aula (2/2)

- O Terraform provisiona recursos de forma declarativa (providers, state, plan/apply) e o state file é crítico e sensível.
- Provisionamento e gestão de configuração são camadas complementares.
- Os segredos nunca vão para o repositório em texto simples.
- O CI/CD automatiza o teste e a aplicação de alterações, com o Git como fonte de verdade.

Leitura Complementar

- Morris, K. *Infrastructure as Code*, 2.ª ed., O'Reilly, 2020.
- Brikman, Y. *Terraform: Up & Running*, 3.ª ed., O'Reilly, 2022.
- Hochstein, L. e Moser, R. *Ansible: Up and Running*, 3.ª ed., O'Reilly, 2022.
- Documentação oficial do Ansible (docs.ansible.com) e do Terraform (developer.hashicorp.com/terraform).

Obrigada.

Questões e comentários